Universidade Federal da Paraíba
Centro de Informática
Disciplina: Programação Estruturada
Professor: Gilberto Farias / Luciano Costa

Lista de Exercícios - Arranjos, Strings, Estruturas e Multi-arquivo

Questões Práticas

Observação:

Para todos os programas que requeiram o uso de arrays, recomenda-se a implementação de duas versões:

- Uma versão em que o número de elementos (N) seja definido diretamente no código. Nesse caso, a quantidade de valores a serem inseridos deve ser informada ao usuário na tela (por exemplo: "Digite 5 números inteiros");
- Uma versão em que o número de elementos (N) seja informado pelo usuário. Nesse caso, o tamanho do array deve ser definido dinamicamente.

Questões

1. Implemente um programa que leia N números inteiros. Armazene os valores em um **array**. Determine os **menores e maiores valores** armazenados, calcule a média entre o menor e o maior valor e determine as **posições** (índices) em que esses valores ocorrem.

Ao final, imprima todas as informações calculadas.

2. Em um programa, crie uma matriz 4×4 e preencha-a com valores inteiros (os dados podem ser definidos diretamente no código). Com essa matriz, calcule:

- A soma da diagonal principal;
- A soma da diagonal secundária.

Ao final, informe se as duas somas são iguais ou diferentes.

3. Implemente um sistema que cadastre um número N de alunos. Para cada aluno, armazene: Matrícula, Nome e Nota. O programa deve:

- Permitir o cadastro;
- Listar todos os alunos;
- Informar o aluno com a maior nota.

As informações devem ser armazenadas em um vetor de `struct Aluno`. A impressão deve ser feita por meio de uma função.

4. Implemente um programa que classifique alunos conforme a nota:

- $Nota \geq 8.0$: APROVADO;
- $6.0 \leq Nota < 8.0$: RECUPERAÇÃO;
- $Nota < 6.0$: REPROVADO.

Defina um `enum Situacao` contendo as possíveis situações. O programa deve permitir que o usuário escolha entre inserir uma nota ou encerrar o programa. Caso uma nota seja inserida, ela deve ser passada para uma função que retorne um valor do tipo `Situacao`. O valor retornado deve ser utilizado em um `switch` para imprimir explicitamente a situação do aluno.

5. Utilizando a mesma lógica da questão anterior, implemente um programa que leia N alunos com sua respectiva nota (utilize `struct Aluno` para armazenar os dados dos alunos — nome, nota, situação) e armazene os dados em um vetor de structs. Calcule:

- Percentual de aprovados;
- Percentual de reprovados;
- Média da turma, utilizando uma função que receba o vetor como parâmetro.

Ao final, imprima um relatório com as informações calculadas.

6. Implemente um sistema que armazene produtos com: Código, Nome, Categoria e Preço. As categorias possíveis são ALIMENTO, BEBIDA e LIMPEZA. Defina um `enum` para representar a categoria e utilize uma `struct` para armazenar os dados. O programa deve:

- Cadastrar uma quantidade N de produtos (informado pelo usuário);
- Listar os produtos por categoria;
- Calcular o valor total dos produtos cadastrados.

7. Implemente um programa que leia uma string e verifique se ela é um palíndromo (palavras, frases ou números que, quando lidos de trás para frente, mantêm o mesmo sentido e sequência de letras ou dígitos, ignorando espaços e acentuação). Ao final, imprima "Palindromo" ou "Nao palindromo".

8. Implemente um programa que leia uma senha digitada pelo usuário e a compare com a senha correta "abc123". Ao final, imprima:

- "Acesso permitido", se forem iguais;
- "Senha incorreta", caso contrário.

9. Implemente um programa que leia duas palavras, determine qual possui maior tamanho, copie a maior palavra para uma terceira variável e imprima o resultado.

10. Implemente um programa que solicite ao usuário um número N de coordenadas (x, y). Ao final, aloque dinamicamente uma matriz bidimensional para armazenar as distâncias euclidianas entre os pontos. Utilize a biblioteca `math.h`, empregando as funções `pow(base, expoente)` para o cálculo da potenciação e `sqrt(x)` para o cálculo da raiz quadrada.

Universidade Federal da Paraíba
Centro de Informática
Disciplina: Programação Estruturada
Professor: Gilberto Farias / Luciano Costa

Questões Teóricas

1. Explique por que, ao percorrer um vetor com laço `for`, deve-se utilizar a condição `i < N` e não `i <= N`.

2. Explique por que as expressões abaixo são equivalentes:

```
v[i]
*(v + i)
```

3. Explique:

   (a) O que faz o operador `sizeof`.

   (b) O que é o tipo `size_t`.

   (c) Por que `size_t` é um inteiro sem sinal.

4. Dado o código:

```
int ar[] = {-1, 2, -2, 7, -5};
size_t tamanhoArray = sizeof(ar);
size_t tamanhoElemento = sizeof(ar[0]);
```

   Explique como calcular o número de elementos do array.

5. Por que não é possível determinar o número de elementos de um array dentro de uma função usando `sizeof(ar)` quando ele é parâmetro?

6. Explique por que a expressão `p + 3` não significa somar 3 bytes ao endereço armazenado em `p`.

7. Se `int *p` aponta para o primeiro elemento de um array, o que significa a expressão `p + 3`? Ela soma 3 bytes ou 3 posições? Justifique.

8. Se `double *p` aponta para o início de um array de doubles, o que representa:

   (a) `p + 4`

   (b) `*(p + 4)`

   (c) `p2 - p1`

9. Considere:

```
int ar[4] = {-1, 12, 9, 2};
int *p = &ar[0];
```

   Explique por que as expressões abaixo são equivalentes:

   (a) `*(p + i)`

   (b) `ar[i]`

   (c) `*(ar + i)`

10. Explique a diferença entre as declarações abaixo:

```
void MinhaFuncao(float *ar);
void MinhaFuncao(float ar[]);
```

11. Por que é recomendável passar também o tamanho do array como argumento da função?

12. Explique por que não é permitido retornar um array completo em C.

13. Analise o código:

```
float* retornaZumbi() {
        float z[10];
        return z;
}
```

    (a) Defina o conceito de variável zumbi.
    (b) Por que esse código é incorreto?
    (c) O que significa dizer que o ponteiro retornado é um "zumbi"?
    (d) Qual seria uma forma correta de implementar essa função?

14. Explique:

    (a) O que é um bloco de memória.
    (b) O que faz a função `malloc()`.
    (c) O que ocorre quando `malloc()` retorna `NULL`.
    (d) Por que atribuir `NULL` ao ponteiro após `free()`.

15. Por que é recomendável utilizar `sizeof(tipo)` ao chamar `malloc()`?

16. Explique a importância da função `free()` e por que é recomendado fazer:

```
free(p);
p = NULL;
```

17. Ao declarar um array multidimensional como parâmetro de função, por que todas as dimensões, exceto a primeira, devem ser especificadas?

18. Explique como funciona a alocação dinâmica de uma matriz utilizando ponteiros para ponteiros.

19. Explique a diferença entre:

```
char *p = "abc";
char s[] = "abc";
```

    Qual deles pode ter o conteúdo modificado? Justifique.

20. Explique o funcionamento das funções:

    - `strlen`
    - `strcmp`
    - `strcpy`

    Por que não devemos usar `==` para comparar strings?

21. Explique:

(a) Diferença entre arquivos `.h` e `.c`.
(b) O que deve conter um arquivo de cabeçalho.
(c) Por que não devemos incluir um arquivo `.c` dentro de outro.

22. Explique a diferença entre:

    - Escopo de programa
    - Escopo de arquivo
    - Escopo de bloco

    E entre:

    - Duração automática
    - Duração fixa (`static`)

23. Explique o efeito da palavra-chave `static`:

    (a) Em uma variável global.
    (b) Em uma variável local dentro de uma função.
    (c) Em uma função.

24. Explique:

    (a) O que é uma estrutura.
    (b) Diferença entre os operadores `.` e `->`.
    (c) Vantagem de passar estrutura por ponteiro.

25. Explique como declarar e acessar um array de estruturas. Por que estruturas são chamadas de variáveis heterogêneas?

26. Explique:

    (a) Diferença entre `struct` e `union`.
    (b) Por que os campos de uma união compartilham memória.
    (c) Quando é interessante utilizar união.

27. Quais as vantagens de utilizar `enum` em vez de constantes definidas com `#define`?

28. Explique a diferença entre:

```
tRegistro func();
tRegistro* func();
```

    Qual é mais eficiente? Qual cuidado deve ser tomado ao retornar ponteiros?